

Chapter 1

Security Requirements and Planning

Deliverable 1 *BurnVault — Secure Software Development (CY321)*

1.1 Security Requirements

Confidentiality: BurnVault ensures that data remains strictly inaccessible to unauthorized parties and the server itself. This is achieved through a zero-knowledge architecture where all cryptographic operations (AES-256-GCM) occur entirely in the browser using the Web Crypto API, ensuring the server only handles opaque ciphertext.

Integrity: The system must guarantee that information has not been modified during transit or storage. BurnVault utilizes AES-256-GCM authenticated encryption, which provides an authentication tag to verify that the ciphertext remains untampered while residing in the ephemeral Redis store.

Availability: Secure communication services must be available to high-security teams in real-time. By utilizing Socket.io over WSS and an ephemeral Redis storage layer, the platform maintains low-latency message relay until the data is consumed or its TTL expires.

Authentication: Access to the relay and storage layers is restricted to verified users. BurnVault implements JWT-based session management and mandatory TOTP-based 2FA, ensuring that only authenticated recipients can access encrypted payloads.

Non-Repudiation: The system establishes cryptographic accountability for every transaction. By employing ECDH P-384 key exchange to derive shared secrets, BurnVault ensures that only the specific private key of the recipient can decrypt the wrapped Content Encryption Key (CEK).

Data Minimisation & Ephemerality: Guided by the "burn-on-read" axiom, BurnVault stores the absolute minimum data required for delivery. Upon a read-receipt or download, the server executes a hard DEL command in Redis, and client-side plaintext buffers are zeroed out to prevent data recovery.

1.2 Risk Management

The following Risk Register outlines the proactive technical mitigations designed to maintain BurnVault’s zero-knowledge posture even in the event of an infrastructure-level compromise.

Risk	Likelihood	Impact	Mitigation Strategy
Server Compromise	Medium	Low	Zero-knowledge design ensures the server holds no plaintext or private keys.
Man-in-the-Middle	Low	High	Enforcement of TLS 1.3, HSTS, and E2E encryption via Web Crypto API.
Cross-Site Scripting	Medium	High	Use of a strict Content-Security-Policy (CSP) and Django auto-escaping.
Brute Force	Medium	Medium	Implementation of API rate limiting and mandatory TOTP 2FA.
Data Leakage (Cache)	Low	Medium	Usage of Cache-Control: no-store and zeroing of plaintext buffers.

Table 1.1: BurnVault Risk Register

1.3 Threat Modelling

1.3.1 System Architecture Diagram

The diagram below illustrates the data flow between the zero-knowledge clients and the blind relay server components.

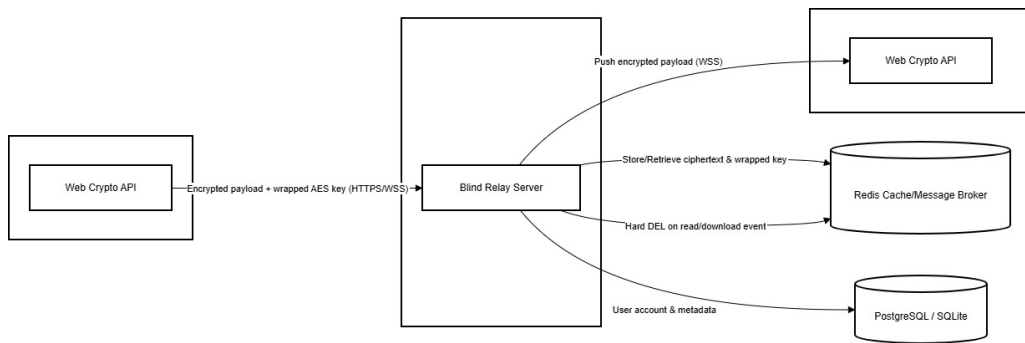


Figure 1.1: BurnVault System Architecture — Data Flow Diagram

1.3.2 STRIDE Threat Model Analysis

The STRIDE model is utilized to verify that every stage of the data flow is protected against common attack vectors.

STRIDE Threat	Mitigation Strategy
Spoofing	JWT-based authentication combined with mandatory TOTP 2FA.
Tampering	AES-256-GCM authenticated encryption for payload integrity.
Repudiation	JWT non-repudiation and session-bound key exchanges.
Information Disclosure	Zero-knowledge server architecture and encrypted Redis storage.
Denial of Service	API rate limiting and Redis connection throttling.
Elevation of Privilege	Django object-level permissions and RBAC.

Table 1.2: STRIDE Analysis and Mitigations

1.3.3 Message Lifecycle and Security Controls

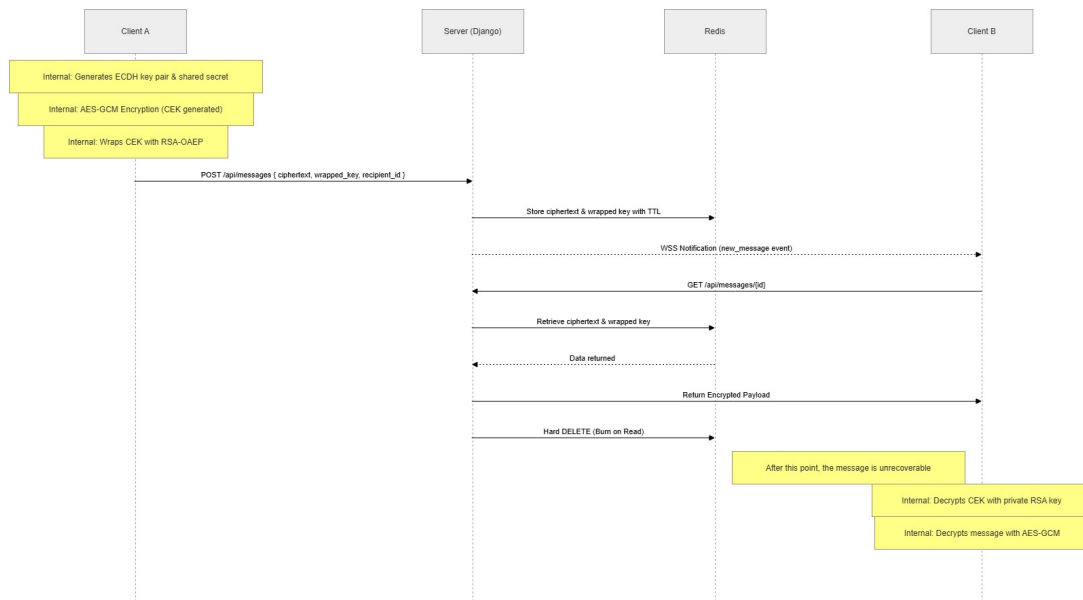


Figure 1.2: BurnVault Message Lifecycle Sequence Diagram

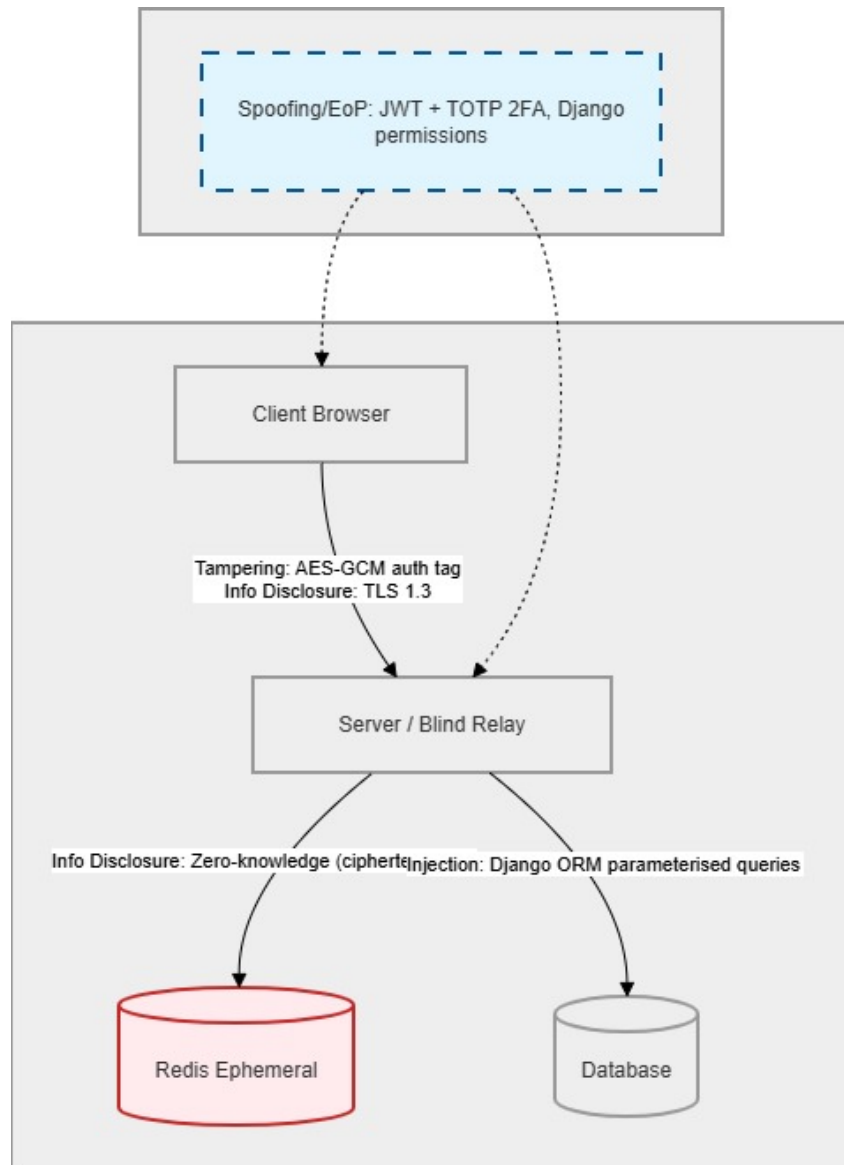


Figure 1.3: Data Flow with STRIDE Mitigations

1. **Cryptography:** Implementation of AES-256-GCM, ECDH P-384, and RSA-OAEP via the Web Crypto API .
2. **Authentication:** Session management via JWT and TOTP multi-factor authentication .
3. **Database Security:** Mitigation of injection via Django ORM parameterized queries.
4. **Network Security:** Secure transport via TLS 1.3, HSTS, and strict CORS policies .